

Problem ID	Captured Flags	Steps
Liars Dice	fsuCTF{800t57r4p_8i11_y0u_4r 3_4_1i4r_4nd_y0u_wi11_5p3nd _4n_37ernity_0n_7hi5_5hip}	1. Random Seed Exploit 2. Figure out how to “win” 3. Find seed and generate win conditions 100x
RSA Master	fsuCTF{You_Can_Do_CRT_For_A ny_E_If_You_Have_Enough_Cip hertexts}	1. RSA 2. Given e 3. Get 7 samples 4. Do math
Jack the Ripper	fsuCTF{John_Work_Well_With_ Rockyou}	1. Use zip2john to get the hash 2. Run rockyou.txt on the hash.txt using john 3. Show the produced hash

Liars Dice

```

Session Actions Edit View Help
Rules: Bid on the total number of dice matching a certain face value.
Bids must increase in quantity, or same quantity with a higher face value.
Format your bids as 'Quantity Face' (e.g., '3 4' for three 4s).
Type 'liar' to challenge the AI's bid.

--- ROUND 1 / 100 ---
Score - You: 0 | AI: 0
Your dice: [3, 6, 3, 6, 2, 1, 4, 5, 1, 4]
$ 1
Invalid format. Please enter 'quantity face' (e.g., '3 4') or 'liar'.
$ liar
You can't call liar on the first turn! Please make a bid.
$ 3 4

AI bids: 3x 6's
Current bid on table: 3x 6's
$ liar

You called Liar! Revealing all dice: [1, 1, 1, 1, 2, 2, 3, 3, 3, 3, 4, 4, 4, 4, 5, 5, 6, 6, 6, 6]
There are 4x 6's on the table.
AI's bid was TRUE. AI wins the round!

```

I first run the program and realize that the AI is “cheating.” After playing for a while I realized that it is impossible to get the right number of dice everytime and every time I get it wrong the AI calls me a liar and I lose. But I do know that `.random()` comes with some caveats that can be beaten. If I can find the seed generated based on the first dice rolls given to me, I can generate and predict every random output and get to the `win()` function.

```

from pwn import *
import random
import time
import re

HOST = 'ctf.cs.fsu.edu'
PORT = 65053

io = remote(HOST, PORT)

start_timestamp = int(time.time() * 1000000)

io.recvuntil(b"Your dice: ")
line = io.recvline().decode().strip()
target_hand = eval(line)

print(f"finding seed near {start_timestamp}... ")

found_seed = None
for offset in range(-2500000, 2500000):
    seed_guess = start_timestamp + offset
    random.seed(seed_guess)

    test_hand = [random.randint(1, 6) for _ in range(10)]

    if test_hand == target_hand:
        found_seed = seed_guess
        print(f"Found Seed: {found_seed}")
        # Note: The AI dice for Round 1 were already rolled right after yours
        ai_hand_round1 = [random.randint(1, 6) for _ in range(10)]
        break

if not found_seed:
    print("Failed to find seed")
    exit()

```

I use the following code to find the given target_hand which in turn gives the seed. I used a Time-Based Brute Forcing attack. First I get the randomly generated dice in the beginning and then I reverse engineer the seed by using the same start_timestamp as the liar_dice.py program and guess the seed in a 5 second window by comparing it to the first dice rolls generated by the liar_dice.py program. The reason I used a 5 second window was because of the latency it takes to get to the server. For every microsecond in that 5-second window, the script: Seeds its own PRNG through random.seed(seed_guess), generates a test hand by rolling 10 dice locally using that seed and compares by checking if the test_hand matches the target_hand from the server. Once we know we are in the right seed we need to immediately call random() to stay in sync with the liar_dice.py program because that is what it does next as well.

```

for round_num in range(1, 101):
    print(f"Solving Round {round_num}")

    if round_num > 1:
        player_hand = [random.randint(1, 6) for _ in range(10)]
        ai_hand = [random.randint(1, 6) for _ in range(10)]
    else:
        player_hand = target_hand
        ai_hand = ai_hand_round1

    all_dice = player_hand + ai_hand

    best_count = 0
    best_face = 0
    for face in range(1, 7):
        count = all_dice.count(face)
        if count ≥ best_count:
            best_count = count
            best_face = face

    bid = f"{best_count} {best_face}"

    io.sendline(bid.encode())

    print(f"Sent bid: {bid}")

    if round_num < 100:
        # Wait for the next round's dice to be printed
        io.recvuntil(b"Your dice: ")
    else:
        io.interactive()

```

For every round (starting at 1 and go to 100) we start with the if else because on the first round the hands are already generated so we can “skip” generating those. Next we need to generate the highest bid possible so that the AI cant win. This is because they can’t call liar AND they cant bid any higher than us so this removes their “cheating” mechanism. We then generate the hands in sync with the program / seed and keep sending the highest possible bid 100 times until the win() function is called.

```

Solving Round 99
Sent bid: 6 1
Solving Round 100
Sent bid: 6 2
[*] Switching to interactive mode
[4, 3, 1, 2, 1, 5, 1, 2, 4, 3]

AI: I call Liar!
Revealing all dice: [1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 4, 4, 4, 4, 5, 6, 6, 6]
There are 6x 2's.
You win the round!

=====
🏆 CONGRATULATIONS! 🏆
You have successfully survived 100 rounds of Liar's Dice!
=====
fsuCTF{800t57r4p_8i11_y0u_4r3_4_1i4r_4nd_y0u_wi11_5p3nd_4n_37ernity_0n_7hi5_5hip}
[*] Got EOF while reading in interactive

```

RSA Master

```
(.venv)-(kali@kali)-[~/CTFs/CTF_Class/cypher/hwB]
$ python3 sol1.py
[*] Opening connection to ctf.cs.fsu.edu on port 65054: Done
[*] Switching to interactive mode
ciphertext = 3663606784420596262723149719826821378811970050165487938997421283051630805320105159214937881353186528774990950051677236187039353431225007756673951556728890
public_key = (7, 7802375855265091303917752730813915230506332521515883516711400678991030840072371790171612481587930504853068638333581649876539140407010291703062259373173521)
[*] Got EOF while reading in interactive
$
[*] Interrupted
[*] Closed connection to ctf.cs.fsu.edu port 65054

(.venv)-(kali@kali)-[~/CTFs/CTF_Class/cypher/hwB]
$ python3 sol1.py
[*] Opening connection to ctf.cs.fsu.edu on port 65054: Done
[*] Switching to interactive mode
ciphertext = 954267165239337721720933499632436257264545782649741901264140529756708823921020687946166668471431612968615381647863411318189850614655317810311298083022891
public_key = (7, 8251938600905071513048356764473682432684867233525106669075382705924651901856861548729127460384256032918787457419904845963083212459986028838246132266978611)
[*] Got EOF while reading in interactive
$
[*] Interrupted
```

We are given an RSA. We are given the ciphertext AND the public key. Something noticeable about the public key is that it gives us e . From this, the exploit lies in the combination of a small public exponent ($e=7$) and the ability to obtain multiple ciphertexts of the same message. I ask gemini to generate the following methods that are necessary to solve the RSA (chinese remainder theorem). It gives the following:

```
def chinese_remainder(n, a):
    sum = 0
    prod = reduce(lambda a, b: a * b, n)
    for n_i, a_i in zip(n, a):
        p = prod // n_i
        sum += a_i * mul_inv(p, n_i) * p
    return sum % prod

def mul_inv(a, b):
    b0 = b
    x0, x1 = 0, 1
    if b == 1: return 1
    while a > 1:
        q = a // b
        a, b = b, a % b
        x0, x1 = x1 - q * x0, x0
    if x1 < 0: x1 += b0
    return x1
```

In order to solve this RSA we need “ e ” or greater samples. In this case we can get 7 samples easily by opening and closing the remote using pwn tools. So I run the following for loop and put the ciphertexts and moduli (what comes after the e ie. “(7, ”) into lists:

```

ciphertexts = []
moduli = []
e = 7

for i in range(e):
    io = remote(HOST, PORT)

    # Parse ciphertext
    io.recvuntil(b"ciphertext = ")
    c = int(io.recvline().strip())

    # Parse public_key = (e, n)
    io.recvuntil(b"public_key = (7, ")
    n = int(io.recvuntil(b")")[:-1])

    ciphertexts.append(c)
    moduli.append(n)
    print(f"Sample {i+1} received.")
    io.close()

```

Then all we have to do is run the chinese remainder theorem on the ciphertexts and the moduli and then calculate the 7th root. We calculate the 7th root through an import called sympy which has a module called integer_nthroot. Then I decode the bytes from hex to get the flag:

```

print("Calculating CRT...")
result_m_exp_e = chinese_remainder(moduli, ciphertexts)

print("Calculating 7th root...")
m, exact = integer_nthroot(result_m_exp_e, e)

flag = bytes.fromhex(hex(m)[2:]).decode()
print(f"\n{flag}")
~

```

When ran:

```

[+] closed connection to server: saved port 65534
Calculating CRT...
Calculating 7th root...

fsuCTF{You_Can_Do_CRT_For_Any_E_If_You_Have_Enough_Ciphertexts}

(.venv)-(kali㉿kali)-[~/CTFs/CTF_Class/cypher/hwB]
$ █

```

Jack The Ripper

```
(kali@kali)-[~/CTFs/CTF_Class/cypher/hwB]
$ file protected.zip
protected.zip: Zip archive data, made by v6.3 UNIX, extract using at least v2.0, last modified Apr 02 2026 22:08:24, uncompressed size 36, method=store

(kali@kali)-[~/CTFs/CTF_Class/cypher/hwB]
$ unzip protected.zip
Archive:  protected.zip
[protected.zip] flag.txt password:
password incorrect--reenter:
password incorrect--reenter:
  skipping: flag.txt                incorrect password

(kali@kali)-[~/CTFs/CTF_Class/cypher/hwB]
```

We are given a tar archive that is password protected. We need to first get the has of the password using zip2john like this:

```
(kali@kali)-[~/CTFs/CTF_Class/cypher/hwB]
$ zip2john protected.zip > hash.txt
ver 2.0 protected.zip/flag.txt PKZIP Encr: cmplen=48, decmplen=36, crc=06B9FFFC ts=B10C cs=06b9 type=0

(kali@kali)-[~/CTFs/CTF_Class/cypher/hwB]
$ ls
hash.txt  protected.zip  sol1.py  sol.py

(kali@kali)-[~/CTFs/CTF_Class/cypher/hwB]
$
```

Now that we have the hash we can run john on the hash using the rockyou.txt file like this to cycle through all the passwords:

```
(kali@kali)-[~/CTFs/CTF_Class/cypher/hwB]
$ john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
No password hashes left to crack (see FAQ)

(kali@kali)-[~/CTFs/CTF_Class/cypher/hwB]
$
```

Then we use john to show the decrypted hash:

```
(kali@kali)-[~/CTFs/CTF_Class/cypher/hwB]
$ john --show hash.txt
protected.zip/flag.txt:Lejoshg:flag.txt:protected.zip::protected.zip

1 password hash cracked, 0 left

(kali@kali)-[~/CTFs/CTF_Class/cypher/hwB]
$
```

And we get that the password is Lejoshg and using that we get the flag:

```
(kali㉿kali)-[~/CTFs/CTF_Class/cypher/hwB]  
$ unzip protected.zip  
Archive:  protected.zip  
[protected.zip] flag.txt password:  
extracting: flag.txt
```

```
(kali㉿kali)-[~/CTFs/CTF_Class/cypher/hwB]  
$ cat flag.txt  
fsuCTF{John_Work_Well_With_Rockyou}
```

```
(kali㉿kali)-[~/CTFs/CTF_Class/cypher/hwB]  
$ █
```